Generative Project Milestone1

Group 3
Student 1 : Jongmin Lee
Student 2 : Heewon Myeong

lee.jongm@northeastern.edu
myeong.h@northeastern.edu

Github repository :
https://github.com/ljmljm7/ie7615-generative-project

Percentage of Effort Contributed by Student1 : 50%

Percentage of Effort Contributed by Student2 : 50%

Signature of Student 1 : Jongmin Lee

Signature of Student 2 : Heewon Myeong

Submission Date : November 11, 2026

1. Introduction

This project aims to build an image captioning model that generates a sentence for an unseen photo. Unlike a classifier that picks from a fixed label set, the model will generate the caption one token at a time.

We will follow the prefix-conditioning approach of ClipCap. A pretrained CLIP encoder turns each image into a 512-dimensional embedding, a small mapping network will convert that embedding into a short sequence of GPT-2 input vectors, and a pretrained GPT-2 decoder will continue the sequence as a caption. Since only the mapping network will be trained, the model can be built from a few thousand image–caption pairs on a single Colab GPU.

In Milestone 1, we selected and preprocessed the dataset, set up the environment (PyTorch, Hugging Face Transformers, and CLIP), built a baseline notebook for the image-to-embedding pipeline, and tokenized the captions with the GPT-2 tokenizer. The result is a cleaned set of **5,000 COCO image–caption pairs**, **1,000 CLIP image embeddings**, and GPT-2 token ids for every caption. Training and caption generation will follow in Milestone 2.

2. Dataset

We prepared **5,000 image–caption pairs** (1,000 images × 5 captions) from COCO Captions val2017 and split them by image into **800 / 100 / 100 images** for train / validation / test.

1) Choice of dataset :

COCO val2017 contains 5,000 images, each with at least five human-written captions (25,014 captions in total). We chose it over Flickr30k for three reasons: its size already matches the 1k–5k target, it downloads directly from the official site (about 1 GB of images plus one annotation file), and COCO is the standard benchmark for image captioning, including for ClipCap. We used val2017 as our entire pool and re-split it ourselves, independent of the official COCO splits.

2) Preprocessing pipeline :

1. Paired images and captions through *image_id*, which links the *images* and *annotations* lists of *captions_val2017.json*.
2. Cleaned each caption: trimmed and collapsed whitespace, lowercased, removed spaces before punctuation, and dropped the trailing period. This will keep GPT-2 from learning arbitrary casing and end-of-sentence variants; the end of a caption will be marked by the end-of-sequence token instead.
3. Removed captions shorter than 5 or longer than 30 words, and duplicate captions of the same image.
4. Opened every image once to detect corrupted files (none were found) and flag grayscale images. Grayscale images were kept because the CLIP processor converts them to RGB.

5. Kept images with at least 5 valid captions, randomly selected 1,000 of them (seed 42), and kept the first 5 captions of each by annotation id, so every image has the same number of reference captions.
6. Split the data by *image_id*, not by row, so all captions of an image fall in the same split. An overlap check confirmed that no image appears in two splits.

< Table 1. Data Filtering Stages >

Stage	Pairs	Images
Raw val2017	25,014	5,000
After length filter (−11)	25,003	5,000
After duplicate removal (−14)	24,989	5,000
After image check (0 unreadable)	24,989	5,000
Images with ≥ 5 valid captions	—	4,979
Final subset	**5,000**	**1,000**

3) Final splits :

< Table 2. Final Splits >

Split	Images	Pairs
Train	800	4,000
Validation	100	500
Test	100	500
Total	**1,000**	**5,000**

Captions average 10.4 words. After GPT-2 tokenization they average 11.1 tokens with a maximum of 34; a 20-token limit covers 99.0% of captions and 30 tokens covers 99.9%. The cleaned table was saved as *coco_captions_clean.csv* (columns image_id, file_name, caption, split plus metadata), with all counts recorded in *data_summary.json*.

3. Proposed Model

The proposed model will condition a pretrained GPT-2 on each image through a learned prefix, and **only the mapping network** that produces the prefix will be trained (Figure 1). The CLIP part is already in place from Milestone 1; the mapping network and GPT-2 decoder will be built in Milestone 2.

1) Architecture :

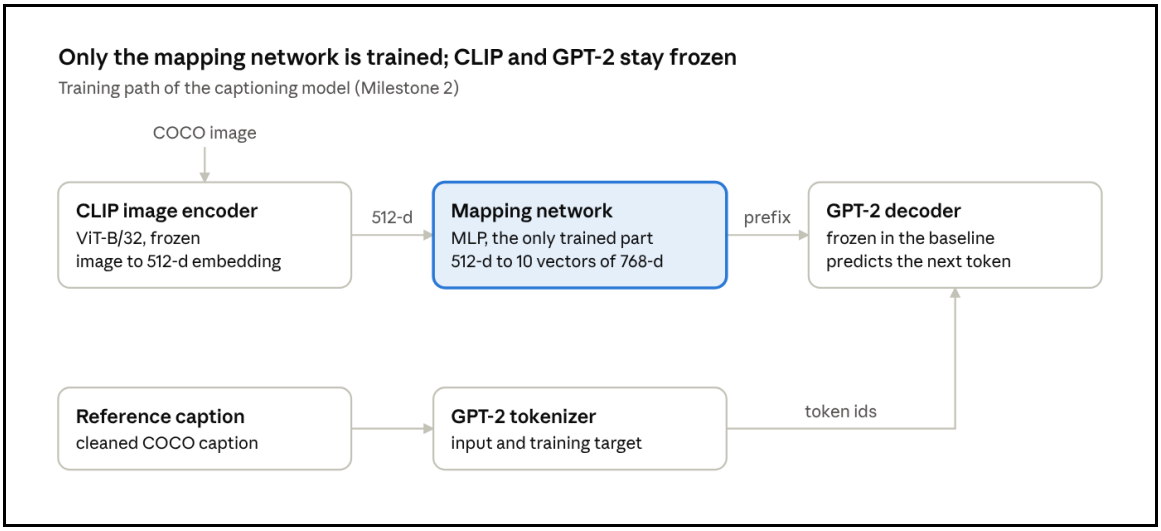
Image encoder: CLIP ViT-B/32, frozen, maps an image to a 512-dimensional embedding. CLIP was trained on image–text pairs, so its embedding already captures what a caption would describe.

Mapping network: a small multilayer perceptron will map the 512-dimensional embedding to a prefix of 10 vectors, each 768-dimensional, the width of GPT-2's token embeddings. The prefix length of 10 follows ClipCap and is a hyperparameter.

Language decoder: GPT-2 will read the prefix as if it were 10 words and continue it with the caption. We will start with GPT-2 frozen and may fine-tune it if captions are weak.

We chose prefix conditioning rather than cross-attention. Cross-attention would add new, randomly initialized layers inside GPT-2, which would need more data than our 4,000 training pairs; the prefix leaves GPT-2 unchanged.

< Figure 1. Model Architecture: Prefix Conditioning >



2) Training objective :

The input sequence will be the 10 prefix vectors followed by the reference caption tokens and an end-of-sequence token. With teacher forcing, GPT-2 will predict each next caption token, and the loss will be the cross-entropy against the reference. Prefix and padding positions will be excluded from the loss (label −100). Training will use the train split, and the validation split will track validation loss.

3) Generation and evaluation plan :

At inference, GPT-2 will receive only the prefix and generate tokens until the end-of-sequence token. Milestone 2 will compare greedy decoding, beam search, and nucleus sampling. For evaluation we plan BLEU and CIDEr against the five reference captions per test image, plus CLIPScore, which scores image–caption agreement without references.

4. Milestone 1 Results

The full pipeline ran end to end in one Colab notebook: all **1,000 images** were embedded into a **1,000 × 512 tensor**, and CLIP matched **all 5 sample images** to their own captions.

(a) Environment setup : we set up Google Colab with an NVIDIA T4 GPU, Python 3, PyTorch 2.14.1 (CUDA build), and Hugging Face Transformers 5.22.1. Pretrained weights were loaded from the Hugging Face Hub: *openai/clip-vit-base-patch32* for CLIP and *gpt2* for the tokenizer.

(b) Image-to-embedding pipeline : each image was resized and normalized by the CLIP processor, passed through the ViT-B/32 vision encoder, and projected to a 512-dimensional embedding. The 1,000 subset images were processed in batches of 32 and saved with their image ids in *clip_image_embs.pt*, so Milestone 2 can train without re-running CLIP. Because CLIP stays frozen, all splits were embedded together; the split column will select the training images later.

(c) Sample 5 image–caption test runs : as a sanity check that the embeddings carry image meaning, we embedded 5 images and one reference caption per image with CLIP, then let CLIP pick the most similar of the 5 captions for each image by cosine similarity. CLIP picked the **correct caption for 5 of 5 images** (Table 3, Figure 2). This is a retrieval check, not an evaluation of generated captions.

< Table 3. Sample 5 Image–Caption Test Runs >

Image ID	Reference caption	CLIP's pick	Match
179765	A black Honda motorcycle parked in front of a garage.	Same caption	Yes
190236	An office cubicle with four different types of computers.	Same caption	Yes
331352	A small closed toilet in a cramped space.	Same caption	Yes
517069	Two women waiting at a bench next to a street.	Same caption	Yes
182417	A beautiful dessert waiting to be shared by two people	Same caption	Yes

< Figure 2. Sample 5 Image–Caption Test Runs: Each Image with Its Reference Caption >



(d) GPT-2 tokenization : all 5,000 captions were converted to GPT-2 token ids, which will serve as both the decoder input and the training target in Milestone 2. For example, "A small closed toilet in a cramped space." becomes 9 tokens: A, small, closed, toilet, in, a, cramped, space, . (ids 32, 1402, 4838, 16146, 287, 257, 43295, 2272, 13). GPT-2 marks a leading space as part of the token, which is why spacing was normalized during cleaning.